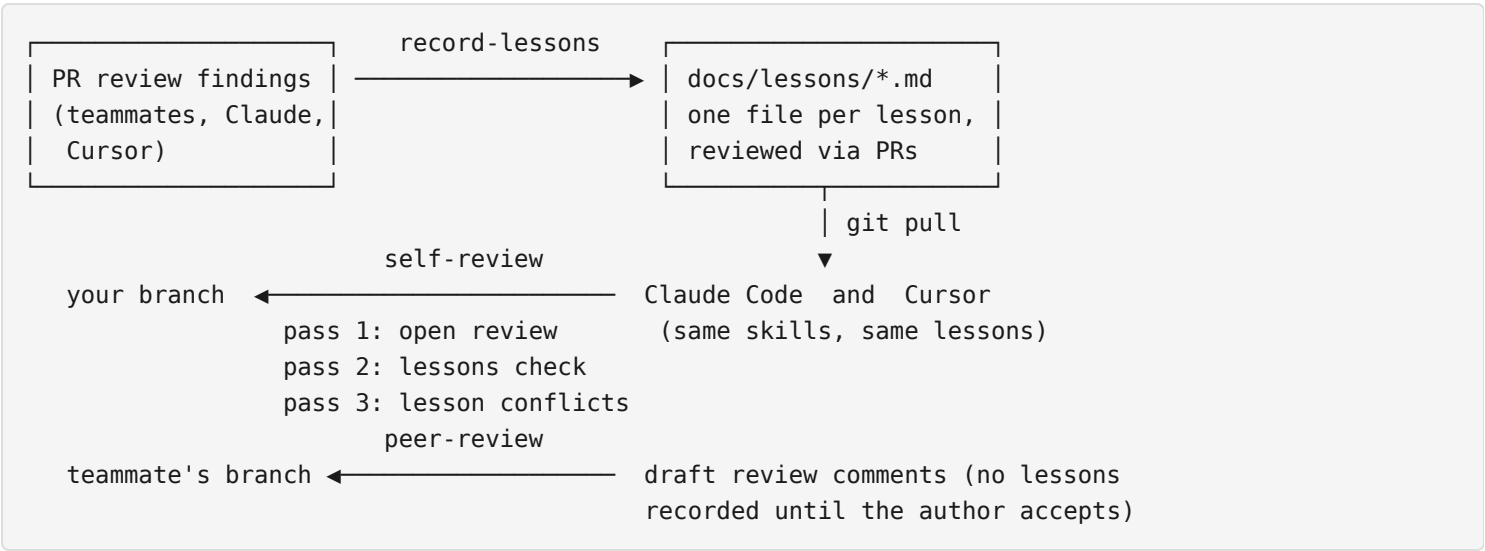


Shared AI Review Lessons: Setup Guide

For: engineering teams that use Claude Code and Cursor on the same repositories.

Goal: Turn PR review findings into shared, versioned lesson files that both Claude Code and Cursor read. Everyone runs a self-review against them before opening a PR, and reviewers use them when reviewing teammates' branches, so past findings aren't repeated. Each tool still does its own open review, so you keep two independent views.

1. How it works



- **Single source of truth.** Lessons live in the repo, not in anyone's personal AI memory.
- **Shared automatically.** Lessons merge through normal PRs, and everyone gets them with `git pull`, whichever tool they use.
- **Keeps learning.** Every review adds or refines lessons. Contradictions are scoped or replaced instead of piling up (section 15), and a periodic cleanup keeps the set small and relevant.
- **Independent reviews stay independent.** Reviews do an open pass *before* reading any lessons.
- **Gated.** Only confirmed findings become lessons, nothing is written or posted without your approval, and the skills behave differently on your branch vs. a teammate's (section 14).

2. Repository structure

```
CLAUDE.md                # main agent instructions (Claude Code reads this)
AGENTS.md -> CLAUDE.md   # symlink, so Cursor reads the same instructions
docs/lessons/
  README.md               # lesson format + writing rules
  2026-09-api-validate-request-body.md
```

```
...
.claude/skills/                # Claude Code reads natively; Cursor reads it for compatibility
  record-lessons/SKILL.md      # review findings → lesson files
  self-review/SKILL.md         # review + fix YOUR branch before opening a PR (with gates)
  peer-review/SKILL.md         # review a TEAMMATE's branch/PR (draft comments only)
  harvest-lessons/SKILL.md     # one-time bootstrap from past PRs, memories, rules
```

Why this works for both tools

- **Skills:** Cursor loads skills from `.claude/skills/` as well as its own folders, so one folder serves both tools.
- **Instructions:** Cursor reads `AGENTS.md` and Claude Code reads `CLAUDE.md`. The symlink makes them the same file.

Windows teammates: Git symlinks need `git config core.symlinks true` and Windows Developer Mode. If that isn't possible, make `AGENTS.md` a normal file containing only:

```
Read and follow CLAUDE.md in this folder. It is the source of truth for agent instructions.
```

3. Setup steps (one person does this once per repo)

```
git checkout -b chore/shared-ai-lessons
mkdir -p docs/lessons .claude/skills/record-lessons .claude/skills/self-review .claude/skills/peer-review .claude/skills/harvest-lessons
ln -s CLAUDE.md AGENTS.md          # skip on Windows; use the pointer file above
```

Then:

1. Create the files in sections 4 through 9 with the contents shown.
2. Add the section 5 block to your existing `CLAUDE.md`. Don't replace the file.
3. If your default branch isn't `main`, replace `main` in the skills with its name.
4. Optionally, add the drift check from section 10 to CI.
5. Open a PR, get it reviewed and merged.
6. Run the harvest from section 11 so the team doesn't start with an empty lessons folder.

4. docs/lessons/README.md

```
# Lessons
```

```
Each file is one lesson learned from a code review. Claude Code and Cursor both read these during `self-review`. Lessons are reviewed in PRs like code.
```

```
## File name
```

```
`YYYY-MM-<area>-<short-kebab-name>.md`, e.g. `2026-09-api-validate-request-body.md`
```

```

## Format
```markdown

title: Validate request bodies at the API boundary
paths: ["src/api/**"] # globs this lesson applies to; ["**"] if global
tags: [validation, security]
severity: high # high | medium | low
source: "PR #42 review" # where it came from
found_by: teammate # claude | cursor | teammate | <name>
added: 2026-09-14
updated: 2026-09-14
occurrences: 1 # bump when the same finding shows up again
applies_when: "" # optional: condition under which the rule holds, e.g. "real-time
sync features"
status: active # active | disputed (disputed = advisory only, not enforced)
supersedes: "" # optional: file name of an older lesson this one replaces

Rule
One or two sentences, phrased as a general rule (not a one-off fix).

Why
What went wrong, or could go wrong. Reference the real incident briefly.

How to check
What a reviewer (human or AI) should look for in a diff to catch this.
```

## Writing rules
- One rule per file. If two lessons say the same thing, merge them and bump `occurrences`.
- General, not specific: "validate request bodies", not "fix line 40 of users.ts".
- Describe code problems, never people. No names in Rule/Why.
- No secrets, credentials, customer data, or internal URLs.
- Delete lessons that no longer apply (code removed, rule enforced by a linter, etc.).
- Contradictions are resolved in the lessons themselves, not in meetings: scope both lessons with
  `applies_when`/`paths` if both are right in different contexts; replace the old one
  (`supersedes`)
  if it is obsolete; mark both `status: disputed` if the evidence doesn't decide.

```

5. Add to `CLAUDE.md`

Add this block to your existing `CLAUDE.md`:

```

## Review lessons (shared by Claude Code and Cursor)
- Team review lessons live in `docs/lessons/`, one file per lesson. See `docs/lessons/README.md`.
- Never save PR review findings to personal memory. Record them in `docs/lessons/` with the
  `record-lessons` skill so the whole team (and Cursor) can use them.
- Before opening a PR, run the `self-review` skill.
- When reviewing someone else's branch or PR, use the `peer-review` skill, not `self-review`.
- When you learn something durable about this codebase from a review, propose a lesson for it.

```

6. `.claude/skills/record-lessons/SKILL.md`

```
---
name: record-lessons
description: Turn PR review findings into shared lesson files in docs/lessons/. Use after a PR
review is received or given, when the user says "record lessons", "save this finding", "remember
this for next time", or pastes review comments.
---

# Record lessons from a review

1. Get the findings. Use, in order of preference:
  - review comments the user pasted;
  - a PR number/URL: fetch its review comments and review summaries (e.g. `gh pr view <n> --
comments`
    and `gh api repos/{owner}/{repo}/pulls/<n>/comments`);
  - findings from the current conversation.
2. Acceptance gate. Only confirmed findings can become lessons. A finding is confirmed when:
the author fixed it in a later commit, the author or reviewers agreed in the thread, or the user
is the author and confirms it. Skip findings that were rejected, are still open/unanswered, or
came from a review whose PR is still under discussion. Say which findings were skipped and why.
3. Filter. Of the confirmed findings, keep only those that would apply to future code. Skip
pure
  style nits already enforced by linters and one-off typos.
4. Generalize. Rewrite each kept finding as a general rule using the format in
`docs/lessons/README.md`.
  Set `paths` to the narrowest globs that cover where this could recur.
5. Deduplicate. Read the existing lessons in `docs/lessons/`. If one already covers the rule,
update it instead (add the new source, bump `occurrences`, refresh `updated`, sharpen "How to
check").
  If a new finding contradicts an existing lesson, don't add it alongside. Classify the conflict
(conditional / obsolete / unclear) the same way `self-review` does, and propose the scoped,
superseding or disputed versions instead.
6. Approval gate. Show the proposed new/updated lesson files and wait for the user's approval
before writing anything.
7. Write new files as `docs/lessons/YYYY-MM-<area>-<short-name>.md`. Set `found_by` to who
caught it
  (`claude`, `cursor`, `teammate`, or a name).
8. Check for sensitive content. No names in Rule/Why, no secrets, no customer data.
9. Report a short list: created / updated / superseded / disputed / skipped (with reason).
Where to commit: in your own PR, or a small separate `chore: lessons` PR from `main`.
  Never commit lessons to someone else's branch.
```

7. `.claude/skills/self-review/SKILL.md`

```
---
name: self-review
description: Review and fix the current branch before opening a PR - an independent bug review
first, then a check against the team's lessons in docs/lessons/, then fixes for clear-cut findings
```

and a report of what was fixed and what needs the user's decision. Use when the user says "self review", "review my branch", "check before PR", or is about to open a PR.

Self-review before a PR

Do the passes in order. Do NOT read docs/lessons/ until pass 1 is finished. This keeps the open review independent instead of turning it into a checklist.

Gate 0 - Is this your branch?

- Compare the authors of the branch's commits (``git log origin/main...HEAD --format='%an <%ae>`'`) with the user (``git config user.email``). If a PR exists, also compare the PR author (``gh pr view --json author``) with the logged-in user (``gh api user --jq .login``).
- If the branch is someone else's, STOP and tell the user to use the ``peer-review`` skill instead.
- If authors are mixed (e.g. you are finishing a teammate's work), ask the user which mode to use.

Setup

- Base branch: ``main`` (change if the repo uses another).
- ``git fetch origin main`` then diff with ``git diff origin/main...HEAD`` and list files with ``git diff --name-only origin/main...HEAD``. Include uncommitted changes (``git diff``) if any.

Pass 1 - Open review (no lessons yet)

Read the diff and enough surrounding code to understand it. Look for:

- correctness bugs, logic errors, off-by-one, wrong conditions
- unhandled errors, null/undefined, edge cases, empty inputs
- security: auth/authz, injection, unvalidated input, secrets, unsafe data exposure
- concurrency, race conditions, transactions, idempotency
- performance traps (N+1 queries, unbounded loops/queries)
- missing or weak tests for the changed behavior

Record findings with file:line, why it is a problem, and a suggested fix.

Pass 2 - Lessons check

1. List ``docs/lessons/*.md`` (skip README.md). Read the frontmatter of each.
2. Select lessons whose ``paths`` match any changed file, or whose ``tags`` clearly relate to the change.
Always include ``severity: high`` lessons with ``paths: ["*"]``.
3. For each selected lesson, apply its "How to check" to the diff. If it has ``applies_when``, first decide whether the changed code meets that condition; skip the lesson if it doesn't.
4. Record violations with file:line and the lesson file name. Violations of ``status: disputed`` lessons are reported as advisory, not as violations.

Pass 3 - Lesson conflicts

If two or more selected lessons would require opposite things for this diff, do not pick one silently.

For each conflict, gather evidence: each lesson's ``source``, ``added``, ``paths``, ``applies_when``, its git history (``git log --follow docs/lessons/<file>``), whether the code or APIs it mentions still exist,

and which rule the current code in each affected area actually follows. Then classify:

- ****Conditional**** - both are right in different contexts (different features, paths, or situations).

Propose an `applies\_when` condition and/or narrower `paths` for each, so they no longer overlap.

- **Obsolete** - one has been replaced: it is older, the code/API it describes is gone, or recent code consistently follows the other rule. Propose deleting or rewriting the old one, with `supersedes: <old-file>` on the surviving lesson.
- **Unclear** - the evidence doesn't decide. Propose `status: disputed` on both, so they are advisory until someone resolves it, and say what evidence would settle it.

Never edit lesson files during self-review. Show the proposals; apply them only if the user approves, then suggest committing them in a `chore: lessons` PR.

Fix

Sort every finding from passes 1 and 2 into one of two groups:

- **Fix now** - the fix is clear, local, and doesn't change intended behavior: a bug with an obvious correction, a missing null check, missing validation, an active lesson violation with a clear remedy, a missing test for the changed code. Apply these fixes.
- **Needs the user's decision** - anything where a reasonable person could choose differently: changes to behavior, APIs, data or schema; design or architecture trade-offs; large or multi-file refactors; fixes you are not confident in; findings tied to `disputed` lessons; and all lesson conflict proposals from pass 3. Do NOT apply these. Describe the options and recommend one.

Rules for fixing:

- Keep each fix minimal - only what the finding needs. No unrelated cleanups.
- Leave the fixes uncommitted so the user can review the diff (`git diff`). Commit only if asked.
- After fixing, run the repo's fast checks (lint, typecheck, tests for the changed code) and re-apply pass 2 to the fixed code. If a fix breaks a check, revert that fix and move it to "Needs the user's decision".
- Never edit `docs/lessons/` during self-review.

Report

1. **Fixed** - for each: the finding (file:line), the lesson it came from if any, and what was changed and why.
2. **Needs your decision** - for each: the finding, the options, your recommendation, and what happens if left as is.
3. **Lesson conflicts** - the lessons involved, the classification, the evidence, and the exact proposed change to each lesson file.
4. **Verification** - which checks ran after the fixes, and their results.
5. **Lessons checked** - count, and which were relevant.
6. **PR readiness verdict** (Gate 1), based on the code *after* fixes.
7. **Candidate new lessons** - findings that look like recurring patterns (Gate 2).

Gate 1 - Ready to open a PR?

- Evaluate after the fixes are applied.
- **NOT READY** if any of these are still unresolved: a high-severity pass-1 finding, or a violation of an `active` lesson with severity high or medium (for example, one waiting in "Needs your decision").
- **READY** otherwise (low-severity items and advisory/disputed lessons don't block).

- The user can override NOT READY with a reason. Suggest noting that reason in the PR description so reviewers see it.

Gate 2 - Recording lessons

- Never call `record-lessons` automatically. A candidate lesson is offered only after the finding is confirmed (the user keeps the fix, or agrees the finding is real), and is recorded only with the user's approval.
- Lesson conflict fixes from pass 3 follow the same rule: proposed, then applied only on approval.

8. `.claude/skills/peer-review/SKILL.md`

```
---
name: peer-review
description: Review a teammate's branch or PR using the team's lessons - produces draft review
comments, never edits their code or records lessons. Use when the user says "review PR #123",
"review <name>'s branch", "code review this PR", or asks to review a branch they didn't write.
---

# Peer review of a teammate's branch

## Gate 0 - Is this someone else's branch?
- Check the PR author (`gh pr view <n> --json author`) or the branch's commit authors against the
user
  (`gh api user --jq .login`, `git config user.email`).
- If it is the user's own branch, STOP and tell them to use `self-review` instead.

## Get the diff (read-only)
- Prefer not to switch the user's working branch. Fetch and diff remotely:
  `git fetch origin main <branch>` then `git diff origin/main...origin/<branch>`
  (or `gh pr diff <n>`). Only check it out (`gh pr checkout <n>`) if the user agrees, e.g. to run
  tests.

## Review
Run passes 1-3 exactly as described in `.claude/skills/self-review/SKILL.md`
(open review first, then lessons check, then lesson conflicts), against this diff.

## Output - draft review comments
- One comment per finding: file:line, severity, what's wrong, suggested fix. Cite the lesson file
  when
    a finding comes from pass 2 so the author can read the rule.
- Group as: blocking (high severity / active lesson violations) and non-blocking (low, advisory,
  disputed).
- Write in a neutral tone about the code, not the person.

## Gates
- No edits: never modify, commit to, or push to the teammate's branch.
- No posting without approval: show the draft comments first. Post to the PR only if the user
  asks.
- No lessons yet: findings are unconfirmed until the author responds. Do not call `record-
  lessons`
```

```
now. Tell the user: once the PR is resolved, run "record lessons from PR #<n>" - `record-lessons` will keep only the findings the author fixed or agreed with.  
- Lesson conflicts from pass 3 go into a separate `chore: lessons` PR from `main` (with the user's approval), never into the teammate's PR.
```

9. `.claude/skills/harvest-lessons/SKILL.md` (bootstrap, so you don't start empty)

```
---  
name: harvest-lessons  
description: Bootstrap docs/lessons/ from existing knowledge - past PR review comments, exported AI memories, and existing rule files. Use when the user says "harvest lessons", "bootstrap lessons", or pastes exported memories/rules to convert.  
---  
  
# Harvest lessons  
  
Sources (use whichever the user provides or asks for):  
  
## A. Past PR reviews in this repo  
1. List recently merged PRs, e.g. `gh pr list --state merged --limit 100 --json number,title,author`  
   (ask the user how far back; default the last 100 merged PRs).  
2. For each, fetch review comments and review bodies:  
   `gh api repos/{owner}/{repo}/pulls/<n>/comments` and `gh api repos/{owner}/{repo}/pulls/<n>/reviews`.  
3. Ignore bot noise, approvals with no content, and resolved-as-wontfix threads.  
4. Group comments that make the same point. A point raised in 2+ PRs is a strong lesson.  
   A single high-severity finding (security, data loss) also qualifies.  
  
## B. Exported memories / notes (Claude, Cursor, personal notes)  
The user pastes text or points at a file. Extract only items about THIS codebase's review findings, conventions, or pitfalls. Ignore personal preferences and anything about people.  
  
## C. Existing rule files  
Read `.cursor/rules/*`, `.cursorrules`, `CONTRIBUTING.md`, style guides, and any team review checklists.  
Convert review-relevant rules that aren't already enforced by linters.  
  
## Then  
1. Draft lessons using the format in `docs/lessons/README.md` (`source` = PR numbers or "harvest: <source>").  
2. Deduplicate against existing lessons and against each other.  
3. Before writing, show the user a table: proposed title | severity | paths | sources | occurrences.  
   Let them drop or edit rows.  
4. Write the approved lessons and suggest one `chore: harvest lessons` PR so the team can review them.
```

10. Optional: drift check in CI

This makes sure `AGENTS.md` stays a symlink to `CLAUDE.md`:

```
[ "$(readlink AGENTS.md)" = "CLAUDE.md" ] || { echo "AGENTS.md must be a symlink to CLAUDE.md"; exit 1; }
```

If you used the pointer-file option for Windows, check that `AGENTS.md` mentions `CLAUDE.md` instead:

```
grep -q "CLAUDE.md" AGENTS.md || { echo "AGENTS.md must point to CLAUDE.md"; exit 1; }
```

11. Harvesting: don't start cold

Do this once after the setup PR merges. Each step ends in a PR, so the team reviews the lessons before they take effect.

11a. From past PR reviews (one person)

Requires the `gh` CLI, logged in with access to the repo. In Claude Code or Cursor, ask:

Use the harvest-lessons skill on the last 100 merged PRs in this repo.

Review the proposed table, approve it, and open the `chore: harvest lessons` PR.

11b. From your own Claude memories (every teammate who saved review findings)

AI memories are personal and live on each person's computer or account. Nobody else can harvest them for you.

1. **Claude Code:** in the repo, run `/memory` to find and open your memory files. The automatic project memory is stored under `~/.claude/projects/`. Then ask:

Go through your memory for this project. Use the harvest-lessons skill to turn every PR review finding and codebase pitfall into lesson files in `docs/lessons/`. Show me the table first. After I approve and the lessons are written, remove those items from your memory.

1. **Claude app (claude.ai):** if you saved findings there, open Settings → memory, copy the relevant entries into a text file, and give them to the harvest skill (step 11c's prompt works).

11c. From Cursor memories and rules (every Cursor user)

1. Copy any review-related entries from Cursor's settings (Rules / Memories / User Rules) into a text file.
2. Ask, in either tool:

Use the harvest-lessons skill on this exported text: `<paste or path>` . Only keep items about this codebase.

- 1. Also check whether the repo has `.cursor/rules/` or `.cursorrules` . The skill can harvest those directly.

11d. Submit

Each person opens a `chore: harvest lessons (<name>)` PR. Reviewers merge duplicates. The skill dedupes, but check anyway.

12. Daily workflow

| When | You say (in Claude Code or Cursor) | What happens |
|---------------------------------------|--|---|
| You get a PR review | "Record lessons from PR #123" | New or updated lesson files to commit |
| You review a teammate's PR | "Peer review PR #456" | Draft review comments for you to edit and post; no lessons yet |
| That PR is resolved or merged | "Record lessons from PR #456" | Lessons from the findings the author fixed or agreed with; commit in a small <code>chore: lessons PR</code> |
| Before opening a PR | "Self review" | Review, fix the clear-cut findings, then report: fixed / needs your decision / verdict |
| Self-review reports a lesson conflict | "Apply the proposed lesson fix" | Lessons scoped, replaced or marked disputed, ready to commit |
| Monthly | "Review docs/lessons: merge duplicates, delete stale ones" | Smaller, sharper lesson set |

Tip: Cursor may pick up skills automatically less reliably than Claude Code. Asking by name ("use the self-review skill") makes it dependable.

13. Keeping Claude and Cursor reviews independent

The shared lessons make both tools catch the same *known* issues, which is intended. To keep their *independent* findings different:

- 1. **Open review first.** Pass 1 runs before any lessons are read, in both `self-review` and `peer-review` .
- 2. **Different model families.** Use a non-Claude model in Cursor (for example GPT or Gemini). If Cursor runs a Claude model, the two reviews end up much more alike.
- 3. **Don't share results early.** Run each review in a fresh chat, and compare only after both are done.

4. **Record who caught it.** The `found_by` field shows which tool or person contributes which lessons. Lessons from one tool's catches teach the other.
-

14. Gates: what runs when, and who decides

The skills behave differently depending on whose branch it is, and nothing permanent happens without a person approving it.

| Gate | self-review (your branch) | peer-review (teammate's branch) |
|-------------------------------|---|--|
| 0. Whose branch? | Checks commit and PR authors. If the branch isn't yours, it stops and points you to <code>peer-review</code> . Mixed authors: it asks | Same check in reverse. If the branch is yours, it points you to <code>self-review</code> |
| Code changes | Fixes clear-cut findings itself (left uncommitted for you to review). Behavior, design or uncertain changes wait for your decision | Never edits, commits to or pushes to their branch |
| Output | What was fixed and how, what needs your decision, check results, and a READY / NOT READY verdict | Draft review comments, blocking vs non-blocking. Posted only if you ask |
| 1. Ready to open a PR? | NOT READY while a high-severity finding or a high/medium active lesson violation is open. You can override with a reason, noted in the PR description | Not applicable |
| 2. Recording lessons | Offered only for findings you've confirmed; written only on your approval | Never during the review. After the PR is resolved, "record lessons from PR #N" keeps only findings the author fixed or agreed with |
| Lesson conflict fixes | Proposed; applied on approval in your PR or a <code>chore: lessons PR</code> | Proposed; applied on approval in a separate <code>chore: lessons PR</code> , never in the teammate's PR |

Why the acceptance gate matters: A reviewer's finding is an opinion until the author confirms it. Recording it at review time would turn unconfirmed or disputed opinions into team rules. Waiting until the PR is resolved means lessons only come from findings that held up. Rejected findings are skipped, and the reason is reported.

The cycle: 1. **Author:** runs `self-review` on their own branch. It fixes the clear-cut findings, and the author decides the rest, then opens the PR. 2. **Reviewer:** runs `peer-review` on the PR and posts the comments they agree with. 3. **Author:** fixes or replies. 4. **Anyone, after the PR is resolved:** runs `record-lessons` on it. Confirmed findings become lessons through a small `chore: lessons PR`. 5. **Everyone:** the next `self-review` and `peer-review` catch those issues automatically.

15. When lessons contradict each other

With several people and two AI tools adding lessons, some will eventually contradict each other. For example, one lesson says "always retry failed API calls" and another says "never retry payment calls". Nobody needs to hold a meeting. The skills catch the contradiction, figure out why, and propose a fix to the lessons. The user approves it, and the fix merges like any other lesson PR.

Who catches it - `self-review` or `peer-review` (pass 3): when two lessons that match the diff ask for opposite things. - `record-lessons`: when a new finding contradicts an existing lesson.

How it decides

| Case | Signals | Proposed fix |
|---|--|---|
| Conditional: both right, in different contexts | Lessons came from different features or paths; the code in each area follows its own rule | Add <code>applies_when</code> and/or narrow <code>paths</code> on both so they stop overlapping |
| Obsolete: one replaced the other | Older lesson; the code or API it mentions is gone; recent code follows only the newer rule | Delete or rewrite the old lesson; add <code>supersedes</code> : to the newer one |
| Unclear: evidence doesn't decide | None of the above fits with confidence | Mark both <code>status: disputed</code> : reported as advice only, not enforced, until someone with context resolves it |

What you see: a "Lesson conflicts" section in the self-review report, listing the lessons involved, the case, the evidence and the exact proposed change. Nothing is edited until you approve.

How it learns: The approved fix is written into the lesson files and merged through a normal `chore: lessons` PR. The next self-review, for anyone on the team, finds scoped or replaced lessons instead of a conflict. Each contradiction is resolved once, by whoever hits it first, and the lessons get more precise over time instead of piling up.

Disputed lessons don't block anyone. They show up as advisory notes. Whoever has the context (often the person who owns that area of code) can resolve one later by approving a scoped or superseding version the next time it comes up.

16. Rules of thumb

- **Repo vs. personal memory:** Facts about the codebase go in the repo. Personal preferences (tone, formatting) stay in personal memory.
- **Lessons are public to the repo:** Anyone with repo access can read them, and they stay in git history. No secrets, customer data or names.
- **Don't duplicate linters:** If a linter or type check can enforce a lesson, add the lint rule and delete the lesson.

- **Keep the set small:** Fewer, sharper lessons beat a long list nobody, human or AI, reads carefully.